QAOA

Genesis Silesky

21 de octubre de 2026

1. QAOA: algoritmo de optimización cuántica aproximada

1.1. Objetivo

Resolver el problema Max-Cut con un circuito variacional de capas alternadas de costo y de mezcla, y comprobar cómo influye el número de capas p.

1.2. Conceptos previos

Concepto previo: VQE y rotaciones ZZ

Ver capítulo 19 (VQE, regla del desplazamiento) y capítulo 12 (rotaciones y acoplamiento $\sigma_z \otimes \sigma_z$). QAOA usa la misma idea híbrida: un circuito parametrizado y un optimizador clásico.

1.3. Fundamento teórico

QAOA fue propuesto por Farhi, Goldstone y Gutmann en 2014. Es un algoritmo híbrido cuántico-clásico para problemas de optimización combinatoria (como el viajante TSP, la coloración de grafos o Max-Cut), aplicables a logística, planificación de producción o diseño de circuitos.

Algoritmo adiabático cuántico (QAA). Si un sistema empieza en el estado fundamental de un Hamiltoniano H_I y $H(t)$ cambia lo bastante despacio hasta H_F, el sistema permanece en el estado fundamental. Se elige H_I con un estado fundamental fácil de preparar y H_F de modo que su estado fundamental codifique la solución. Con $H(t)$ pasando de H_I en $t = 0$ a H_F en $t = T$, la evolución se simula con la fórmula de Trotter–Suzuki, alternando pasos pequeños de H_I y de H_F. En la práctica, el ruido y los límites físicos impiden las evoluciones largas que pide la adiabaticidad.

QAOA. Adopta la misma idea con evoluciones más cortas y rápidas, y busca una solución *aproximada* optimizando parámetros. Los pasos son:

1. **Hamiltonianos:** se define un Hamiltoniano de costo H_C (el estado de mayor energía codifica la solución) y uno de mezcla H_M que no conmuta con H_C (aquí, $H_M = \sum_i X_i$).
2. **Estado inicial:** $|s\rangle = |+\rangle^{\otimes n}$, el estado de mayor energía de H_M.
3. **Circuito:** la capa de costo $U_C(\gamma) = e^{-i\gamma H_C}$ y la de mezcla $U_M(\beta) = e^{-i\beta H_M}$ (que es $R_x(2\beta)$ en cada qubit).
4. **Evolución:** con p capas, $\gamma = (\gamma_1, \ldots, \gamma_p)$ y $\beta = (\beta_1, \ldots, \beta_p)$:

$$|\psi_p(\gamma, \beta)\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n} . \tag{1}$$

5. **Valor esperado:** $E(\gamma, \beta) = \langle \psi_p | H_C | \psi_p \rangle$.

6. **Optimización:** un método clásico actualiza γ y β para *maximizar* E.

Max-Cut. Dado un grafo $G = (V, E)$ con n vértices, se reparten los vértices en dos grupos para *maximizar* el número de aristas entre grupos distintos. Si $x_i \in \{0, 1\}$ dice a qué grupo (por ejemplo, a qué día) pertenece el vértice i, la función a maximizar es

$$L(x) = \sum_{(u,v)\in E} \big(x_u + x_v - 2x_u x_v\big). \tag{2}$$

Para dos lugares turísticos unidos por una arista hay $2^2 = 4$ formas de repartirlos: 00 y 11 cortan 0 aristas y 01 y 10 cortan 1. Para pasar a un Hamiltoniano se sustituye $x_i \to \frac{I - Z_i}{2}$ (vale 0 en $|0\rangle$ y 1 en $|1\rangle$) y queda

$$H_C = \sum_{(u,v)\in E} \frac{1 - Z_u Z_v}{2}. \tag{3}$$

Cómo se hace $e^{-i\gamma Z_u Z_v}$. Con una CNOT, una rotación R_z sobre el segundo qubit y otra CNOT. En el ejemplo, la capa de costo con ángulo de puerta $a = -\gamma$ coincide con $e^{-i\gamma H_C}$ salvo una fase global.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 20 - QAOA: resolver el problema Max-Cut con un circuito variacional

# %% Celda 1: importaciones
import itertools
import numpy as np
from scipy.linalg import expm
from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator, Statevector, SparsePauliOp

# %% Celda 2: el problema Max-Cut
# Dado un grafo, se reparten los vertices en dos grupos (0 o 1) para que haya
# el MAYOR numero posible de aristas entre grupos distintos ("aristas cortadas").
# Ejemplo del PDF: dos lugares turisticos unidos por una arista; x_i dice en
# que dia se visita el lugar i. Funcion a maximizar:
#     L(x) = suma sobre aristas (u,v) de  x_u + x_v - 2 x_u x_v.
aristas = [(0, 1)]
n = 2


def corte(x):
    return sum(x[u] + x[v] - 2 * x[u] * x[v] for u, v in aristas)


for x in itertools.product((0, 1), repeat=n):
    print(f"x = {x} -> aristas cortadas = {corte(x)}")

# %% Celda 3: pasar el problema a un Hamiltoniano
# Se cambia x_i por (I - Z_i)/2 (vale 0 si el qubit esta en |0> y 1 si esta en |1>):
#     Hc = suma sobre aristas de (1 - Z_u Z_v)/2.
# Su valor esperado es el numero promedio de aristas cortadas. Maximizarlo
# resuelve el problema.
def hamiltoniano_costo():
    terminos, coefs = [], []
    for u, v in aristas:
```

```
        terminos += ["I" * n, "".join("Z" if i in (u, v) else "I" for i in range(n))[::-1]]
        coefs += [0.5, -0.5]
    return SparsePauliOp(terminos, coefs).simplify()


Hc = hamiltoniano_costo()
Hc_matriz = Hc.to_matrix()
print("Autovalores de Hc:", np.linalg.eigvalsh(Hc_matriz).round(3), "-> el maximo es", Hc_matriz
    ↪ .diagonal().real.max())


# %% Celda 4: el circuito QAOA
# Estado inicial |+>...|+> (todas las cadenas con la misma amplitud). Cada capa tiene:
#   - capa de costo   e^{-i gamma Hc}: para cada arista, una rotacion ZZ,
#   - capa de mezcla  e^{-i beta sum X_i}: una Rx(2 beta) en cada qubit.
# Aqui los parametros se usan como angulos de las puertas: a = -gamma en la rotacion
# ZZ (puerta Rzz(a) = CNOT, Rz(a), CNOT) y b = 2 beta en la Rx.
def circuito_qaoa(params, desplazar=None):
    """Circuito QAOA. `desplazar` = (parametro, numero_de_uso, delta) mueve el angulo
    de UNA sola puerta (se necesita para el gradiente, ver Celda 5)."""
    usos = {}

    def angulo(i):
        k = usos.get(i, 0)
        usos[i] = k + 1
        if desplazar is not None and desplazar[0] == i and desplazar[1] == k:
            return params[i] + desplazar[2]
        return params[i]

    p = len(params) // 2
    qc = QuantumCircuit(n)
    qc.h(range(n))
    for capa in range(p):
        for u, v in aristas:
            qc.cx(u, v)
            qc.rz(angulo(2 * capa), v)
            qc.cx(u, v)
        for q in range(n):
            qc.rx(angulo(2 * capa + 1), q)
    return qc


# Comprobacion: la capa de costo con angulo a = -gamma es e^{-i gamma Hc}
# (salvo una fase global).
gamma = 0.7
capa_costo = QuantumCircuit(n)
capa_costo.cx(0, 1)
capa_costo.rz(-gamma, 1)
capa_costo.cx(0, 1)
exacta = expm(-1j * gamma * Hc_matriz)
M = Operator(capa_costo).data
fase = (exacta[0, 0] / M[0, 0])
print("La capa de costo coincide con e^{-i gamma Hc} (salvo fase):", np.allclose(exacta, fase *
    ↪ M))

# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
circuito_qaoa([0.5, 0.3]).draw()


# %% Celda 5: el valor esperado <Hc> y su gradiente
def valor_esperado(params, desplazar=None):
    return float(np.real(Statevector(circuito_qaoa(params, desplazar)).expectation_value(Hc)))

```

```python

def gradiente(params, s=np.pi / 2):
    """Regla del desplazamiento de parametros: dE/d(angulo) = (E(+s) - E(-s)) / 2.

    Un mismo parametro aparece en varias puertas (la mezcla usa el mismo beta en
    todos los qubits). La regla vale para UNA puerta a la vez, asi que se suma la
    contribucion de cada puerta donde aparece el parametro."""
    g = np.zeros_like(params)
    for i in range(len(params)):
        usos = len(aristas) if i % 2 == 0 else n
        for k in range(usos):
            g[i] += (valor_esperado(params, (i, k, s)) - valor_esperado(params, (i, k, -s))) / 2
    return g


# Se comprueba contra una derivada numerica.
p0 = np.array([0.4, 0.9])
eps = 1e-6
numerico = np.array([(valor_esperado(p0 + eps * e) - valor_esperado(p0 - eps * e)) / (2 * eps)
    ↪ for e in np.eye(2)])
print("Gradiente (parameter-shift):", gradiente(p0).round(5), "| numerico:", numerico.round(5))


# %% Celda 6: optimizar (maximizar <Hc>) con p = 1 capa
# Ascenso por gradiente:  params <- params + lr * gradiente.
def optimizar(p, lr=0.3, iteraciones=60, semilla=1):
    rng = np.random.default_rng(semilla)
    params = rng.uniform(0, 1, 2 * p)
    for it in range(iteraciones):
        params = params + lr * gradiente(params)
    return params, valor_esperado(params)


params, valor = optimizar(1)
print(f"p = 1: parametros = {params.round(3)} | <Hc> = {valor:.4f} (maximo posible = 1)")

# %% Celda 7: que cadena se mide con mas probabilidad
# Con los parametros optimos se miden casi siempre cadenas que cortan la arista.
probs = Statevector(circuito_qaoa(params)).probabilities_dict()
for cadena, prob in sorted(probs.items()):
    print(f"cadena {cadena}: probabilidad = {prob:.4f} | aristas cortadas = {corte([int(c) for c
    ↪  in cadena[::-1]])}")

# %% Celda 8: cambiar el numero de capas p
# El PDF pide repetir con otros p para ver si el resultado depende de p.
for p in (1, 2, 3):
    _, valor = optimizar(p)
    print(f"p = {p}: <Hc> = {valor:.4f}")
```

1.5. Explicación celda por celda

Celda 2 Lista las cuatro formas de repartir los dos lugares y cuántas aristas corta cada una.

Celda 3 Construye H_C y comprueba que su valor máximo es 1.

Celda 4 Arma el circuito QAOA y verifica que la capa de costo es $e^{-i\gamma H_C}$.

Celda 5 Calcula $\langle H_C \rangle$ y su gradiente. Como un mismo parámetro aparece en varias puertas (la mezcla usa el mismo β en todos los qubits), la regla del desplazamiento se aplica a cada puerta por separado y se suman las contribuciones. Se compara con la derivada numérica.

Celda 6 Ascenso por gradiente con $p = 1$.

Celda 7 Muestra las probabilidades de cada cadena con los parámetros óptimos.

Celda 8 Repite con $p = 1, 2, 3$, como pide la práctica.

1.6. Salida obtenida

```
x = (0, 0) -> aristas cortadas = 0
x = (0, 1) -> aristas cortadas = 1
x = (1, 0) -> aristas cortadas = 1
x = (1, 1) -> aristas cortadas = 0
Autovalores de Hc: [0. 0. 1. 1.] -> el maximo es 1.0
La capa de costo coincide con e^{-i gamma Hc} (salvo fase): True
Gradiente (parameter-shift): [-0.44849  0.08848] | numerico: [-0.44849  0.08848]
p = 1: parametros = [-1.571  0.785] | <Hc> = 1.0000 (maximo posible = 1)
cadena 00: probabilidad = 0.0000 | aristas cortadas = 0
cadena 01: probabilidad = 0.5000 | aristas cortadas = 1
cadena 10: probabilidad = 0.5000 | aristas cortadas = 1
cadena 11: probabilidad = 0.0000 | aristas cortadas = 0
p = 1: <Hc> = 1.0000
p = 2: <Hc> = 1.0000
p = 3: <Hc> = 1.0000
```

(El dibujo del circuito no se reproduce aquí.) Es determinista (la optimización usa una semilla fija).

1.7. Interpretación

El gradiente por desplazamiento coincide con el numérico (-0.44849 y 0.08848). Con $p = 1$ el valor esperado llega a 1.0000 (el máximo): la probabilidad se reparte entre 01 y 10 (0.5 cada una), que son las dos formas de cortar la arista, y 00 y 11 quedan en 0. Con $p = 2$ y $p = 3$ también se obtiene 1.0000: en este grafo tan pequeño, una sola capa alcanza.

1.8. Ejercicios

1. Cambia el grafo a un triángulo (aristas $(0, 1), (1, 2), (0, 2)$, $n = 3$). ¿Cuál es el valor máximo de H_C? ¿Llega QAOA con $p = 1$?

2. Cambia la tasa de aprendizaje y el número de iteraciones y mira cómo cambia el resultado.

3. Prueba otras semillas para los parámetros iniciales con $p = 1$: ¿llega siempre a 1?

1.9. Errores comunes

- Confundir el signo: QAOA aquí *maximiza* $\langle H_C \rangle$ (el número de aristas cortadas), mientras que VQE minimiza una energía.

- Olvidar que los parámetros compartidos necesitan sumar las contribuciones de cada puerta al calcular el gradiente.